Gemini CLI - Concrete Architecture Analysis

March 10, 2026

Authors:
Iram (18ih2@queensu.ca), Lena (22bsn3@queensu.ca), Maciek (20mjb21@queensu.ca), Mack (22NWD3@queensu.ca) Martin (martin.allerdissen@queensu.ca)

Abstract:
This report presents a concrete architectural analysis of Gemini CLI, Google's open-source command-line interface for interacting with the Gemini family of large language models. Building on our Assignment 1 conceptual architecture, we recover the system's as-built structure using SciTools Understand and organize the implementation into a set of concrete subsystems including the CLI Frontend, Core Runtime, Tool Orchestration, Built-in File Tools, Memory, Policy and Confirmation, and MCP Support. The recovered architecture confirms that Gemini CLI is centered on a centralized orchestration model in which the packages/core subsystem coordinates model interaction, tool execution, session context, and safety controls, while the packages/cli subsystem manages terminal interaction and rendering.

Our analysis examines the system at two levels. First, we present the high-level concrete architecture, describing the major subsystem boundaries, their dependencies, and the external interfaces through which Gemini CLI interacts with the Gemini API, the file system, the shell, and external MCP servers. Second, we provide a detailed analysis of the Core Runtime subsystem, showing that what appeared conceptually as a single orchestration component is concretely decomposed into specialized internal elements for configuration, conversation management, tool scheduling, policy enforcement, memory handling and external integration. Two use cases are used to illustrate how these components collaborate at runtime, and a reflection analysis compares the recovered architecture against our earlier conceptual model.

We find that the conceptual architecture from Assignment 1 correctly captured the system's overall centralized control structure and its separation between interface and backend concerns, but the concrete architecture reveals a greater degree of internal specialization than was visible at the conceptual level. In particular, tool execution, safety enforcement, and external integration are implemented through more structured coordination mechanisms than originally modeled. Overall, Gemini CLI's concrete architecture reflects a platform-oriented design that preserves a predictable and auditable control flow while supporting extensibility through modular runtime services and external tool integration.

High-Level Concrete Architecture:

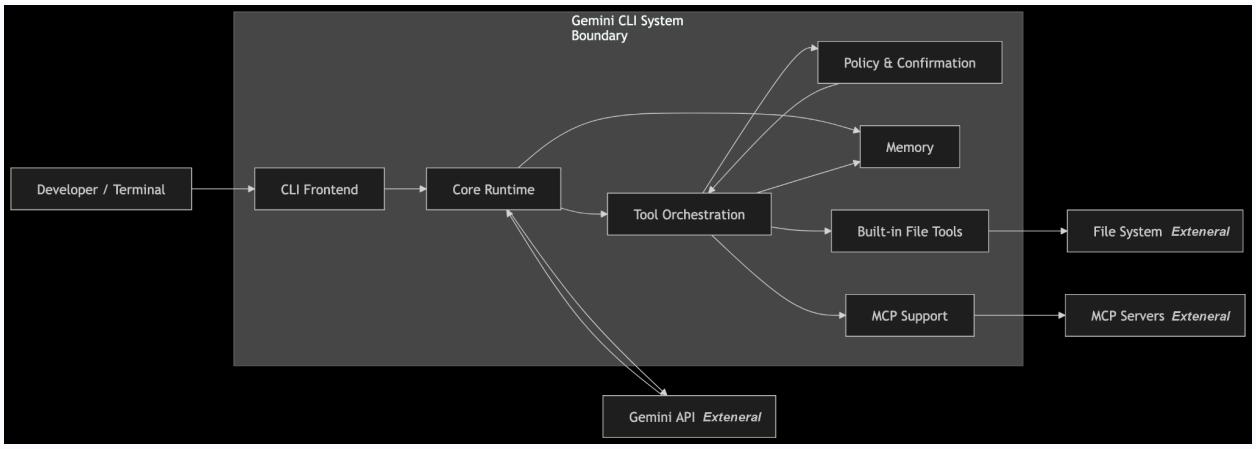
The high-level concrete architecture recovered from the extracted dependency information and refined with Understand is centered on a CLI Frontend, a Core Runtime, and supporting

subsystems for Tool Orchestration, Policy & Confirmation, Memory, Built-in File Tools, and MCP Support. This decomposition keeps the architecture at the subsystem level required by the report while reflecting the major implementation groupings and dependency paths visible in the codebase and dependency graph.

The CLI Frontend represents the terminal-facing part of the system and depends on the Core Runtime because all user requests, streamed responses, and interactive approval flows are routed through the runtime rather than handled directly in the interface. The Core Runtime is the central coordinator: it manages turn processing, session state, model communication, and the overall control flow. It depends on Tool Orchestration when model output requires actions beyond text generation, and it depends on Policy & Confirmation because potentially sensitive operations must be checked or approved before execution proceeds. This preserves the serialized, approval-gated agent loop already identified in the conceptual architecture.

Tool Orchestration represents the dispatch layer that routes action requests to the appropriate execution subsystem. Its dependency on Built-in File Tools exists because local codebase exploration and modification are implemented through internal file-oriented tools that ultimately interact with the File System external interface. Its dependency on MCP Support exists because externally provided tools are reached through MCP servers rather than through built-in implementations. Its dependency on Memory exists because some tool-triggered or turn-level actions update or retrieve persisted project context. Policy & Confirmation is modeled as a separate subsystem rather than folded into Core Runtime because the implementation treats approval and governance as a distinct control boundary around execution-sensitive actions.

Outside the system boundary, Gemini API, File System, and MCP Servers are modeled as external interfaces because they are depended on by the concrete implementation but are not part of the Gemini CLI codebase itself. Overall, this concrete architecture stays close to the frontend/core separation identified in A1, but refines the tool-related part of the system into more explicit execution, governance, memory, and integration subsystems so that the major concrete dependencies are visible and traceable.



Legend: *Rectangles represent major subsystems or components of the Gemini CLI architecture. Solid arrows represent primary dependency or communication relationships between subsystems during request processing. Components inside the grey region are internal to the Gemini CLI system, while components outside the boundary represent external interfaces or services that the system interacts with (e.g., the Gemini API, local File System, and MCP Servers).*

Inner Architecture of packages/core

Overview and Conceptual-Concrete Comparison:

In Assignment 1, packages/core was modeled as a single Core Runtime responsible for prompt construction, model communication, tool coordination, and policy enforcement. The recovered implementation shows that this view was directionally correct but too coarse. In the concrete architecture, packages/core is still the central orchestrator, but it is internally decomposed into several specialized elements responsible for model interaction, tool scheduling, safety enforcement, memory handling, configuration, and external integration.

This means the conceptual architecture captured the correct center of control, but not the degree of internal specialization required by the production implementation. The most important refinement is that tool execution is not a direct one-step dispatch from core to tools. Instead, the concrete system introduces explicit scheduling, approval, and execution responsibilities, making the runtime more structured and safety-oriented than the A1 model suggested.

Layer 1: Facade (GeminiClient):

Conceptual (A1): A1 described core as the single point of entry for CLI requests, receiving user input via an event-driven API and orchestrating all downstream actions. No distinction was made between the external API surface and internal orchestration logic.

Concrete: The entry point is materialized as GeminiClient (packages/core/src/core/client.ts), which acts as a Facade pattern over the entire core subsystem. It is the only class the CLI package directly instantiates. Its concrete methods include: sendMessageStream(message, options) to initiate a turn, generateContent(prompt) acting as a stateless LLM call for side tasks, resetChat() to clear the conversation history and setTools(tools) to update the available tool definitions.

GeminiClient coordinates internal components: GeminiChat (manages API calls and history), Turn (processes request-response cycles), ModelRouterService (selects model per request), ChatCompressionService (handles token overflow), and coreEvents (global event emitter for cross-subsystem signals).

Interaction: The CLI layer only imports and calls GeminiClient. All other core subsystems are hidden behind this facade. This design decision isolates the CLI from internal refactoring; changes to the chat layer, routing logic, or tool execution do not affect the CLI's integration surface.

Layer 2: Chat (GeminiChat, Turn, BaseLlmClient, ContentGenerator)

Conceptual: Implied direct connection between core and Gemini API for prompt construction and response streaming.

Concrete: Four components handle model interaction:

1. GeminiChat:wraps the google/genai SDK. It maintains two parallel histories (comprehensive for persistence, curated for token limits), implements retry logic with exponential backoff

2. Turn: represents a single request-response cycle. It processes event stream, extracts function calls, calls CoreToolScheduler
3. BaseLlmClient: a stateless LLM operations for side tasks (token counting, JSON generation, classification)
4. ContentGenerator: interface with Decorator implementations like LoggingContentGenerator, RecordingContentGenerator, FakeContentGenerator

Architectural Significance: A1 missed dual history structure critical for token management and the composable Decorator stack enabling telemetry without modifying core logic.

Layer 3: Tool Execution — State Machine Architecture

Conceptual (A1): A1 described the Tool Registry as a "central dispatch hub" that routes tool invocations to built-in tools or MCP servers. Tool execution was characterized as a direct operation: core invokes the tool and folds the result back into the next model prompt, with a policy layer gating sensitive operations.

Concrete: The tool execution layer splits across three non-overlapping components:

1. **ToolRegistery:** catalogs tools and utilizes getFunctionDeclarations() to produce JSON schemas sent to the Gemini API. This does not execute tools

2. **CoreToolScheduler:** manages tool call lifecycle as a formal state machine with validating, awaiting approval, scheduled, executing, success, error, and cancelled states. The key methods in this component include the schedule(toolCallRequest) to initiate the lifecycle, validate parameters, the _checkPolicty(toolName, argos) to call the PolicyEngine, _requestConfirmation(toolCall) to publish to the MessageBus, and the _handleUserResponse(outcome) to process approval, cancel, modify requests.

3. **ToolExecutor:** executes a single ToolInvocation after all gates are cleared. Handles live output streaming, tracks subprocess PIDs for shell commands, and applies output truncation.

Architectural Significance: State machines, especially awaiting_approval state and loop-back through policy evaluation, was entirely absent from A1 but is central to safety guarantees.

Layer 4: Safety Subsystem — Three Independent Components

Conceptual (A1): A1 described a single "policy and confirmation layer" sitting between the orchestrator and tool execution, enforcing governance rules. Users configure allowed, denied, or confirmation-required tools.

Concrete: The safety subsystem consists of three collaborating but independent subsystems:

1. **PolicyEngine:** This component evaluates each tool call against the rule set loaded from the TOML files. It determines which tools can and cannot be executed or require user permission.
2. **MessageBus:** Extends Node.js EventEmitter as a pub-sub channel for approval flow. When CoreToolScheduler receives ASK_USER from PolicyEngine, it publishes a ToolConfirmationRequest event carrying SerializableConfirmationDetails. The CLI subscribes to MessageBus, renders the appropriate UI, and publishes ToolConfirmationResponse containing ToolConfirmationOutcome (ProceedAlways, ProceedOnce, Cancel, Modify, ModifyAlways).
3. **HookSystem:** Implements an Observer pattern for lifecycle interception. Hooks fire at seven execution boundaries including: BeforeAgent, AfterAgent, BeforeModel, AfterModel, BeforeToolSelection, BeforeToolExecution, AfterToolExecution.

Hooks load from project-level, user-level, and built-in sources. Implementations may be JavaScript, TypeScript, or shell scripts. HookAggregator combines results using last-write-wins.

Architectural Significance: This is the most significant divergence from A1. What appeared as a single architectural layer is three independent subsystems. PolicyEngine is rule-based with TOML configuration, supporting wildcard patterns and regex argument matching, substantially more expressive than the binary allow/deny A1 implied. The MessageBus was entirely absent from A1; it is the mechanism making approval gates asynchronous without blocking the Node.js event loop. HookSystem was completely accurate at the level of a single agent loop but does not account for nested execution when a parent agent delegates to subagents.

Layer 5: Multi-Agent Framework

GeminiChat sessions with agent-scoped tool whitelists; enforces max_turns, timeout_mins; loops until complete_task called. The DelegateToAgentTool dynamically generates schema from AgentRegistry; model sees agent names as enumerated type The A2AClientManager manages remote agent delegation. Subagent invocation creates nested execution contexts with independent GeminiChat sessions and CoreToolScheduler state.

Architectural Significance: A1 described single-agent system and placed multi-agent in future. Changes system from sequential pipeline to recursive, tree-structured execution model.

Layer 6: Model Routing

Conceptual (A1): A1 made no mention of model routing. It implied static model selection configured at startup.

Concrete: ModelRouterService is called by GeminiClient on every sendMessageStream() invocation to determine which Gemini model to use. Wraps a RoutingStrategy implementation.

Architectural Significance: Model routing was entirely absent from A1. The ClassifierStrategy is particularly significant: the system uses a machine learning classifier to dynamically select between models per request, not a static configuration value. The CompositeStrategy means routing logic is composable and extensible without modifying the core loop.

High Level Reflexion Analysis

This section compares the top-level conceptual architecture from A1 against the concrete high-level architecture derived from Understand dependency analysis. A1 identified six top-level components within the system boundary: CLI Frontend, Core Runtime, Tool Registry, Built-in Tools, and two external dependencies (Gemini API, MCP Servers) connected by five principal dependency edges. The concrete architecture largely validates A1's structural decomposition, but reveals several divergences in component scope, dependency direction, and missing subsystems.

Convergences

The concrete architecture confirms several aspects of A1's model. The fundamental split between packages/cli and packages/core is reflected directly in the repository's monorepo structure and validated by Understand's dependency graph: packages/cli depends on packages/core through a well-defined API surface, and no reverse dependency exists. The Core Runtime's role as the central orchestrator—managing prompt construction, model communication, and tool coordination—is accurate. The external dependency on the Gemini API over HTTPS and the integration with MCP Servers via stdio/SSE/HTTP transports are both confirmed. The general control flow A1 described is also validated as the primary execution path.

Divergence 1: Tool Registry Scope

A1 Conceptual View: A1 described the Tool Registry as a "central dispatch hub" that both catalogs available tools and routes invocation requests to Built-in Tools or MCP Servers. In the conceptual diagram, the Tool Registry has outgoing dependency edges to both Built-in Tools (invoke/toolArgs) and MCP Servers (stdio/HTTP/SSE), positioning it as the component that actively dispatches and executes tool calls.

Concrete Architecture: Understand analysis reveals that the Tool Registry (src/tools/tool-registry.ts) is narrower in scope than A1 described. It catalogs tool definitions and exports getFunctionDeclarations() to produce JSON schemas for the Gemini API, but it does not execute or dispatch tools. Tool dispatch and execution are handled entirely within Core by the CoreToolScheduler and ToolExecutor. This means the dependency edges from "Tool Registry" to "Built-in Tools" and "MCP Servers" shown in A1's Figure 1 are more accurately edges from Core Runtime to those components, mediated by the scheduler and executor subsystems.

Evidence: The Understand dependency graph shows no direct call edges from tool-registry.ts to any tool implementation or MCP client. Instead, coreToolScheduler.ts imports and calls tool-executor.ts, which in turn invokes individual tool implementations. This separation is architecturally significant because it means the catalog (what tools exist) is decoupled from the lifecycle (how and when tools run).

Divergence 2: Configuration Subsystem Absent from A1

A1 Conceptual View: A1 made no mention of a dedicated configuration subsystem at the top level. Configuration was implicitly assumed to be handled within Core Runtime.

Concrete Architecture: The concrete architecture reveals a distinct Configuration subsystem (src/core/config/) that is depended upon by nearly every other subsystem. It loads and merges settings from multiple sources: gemini.toml files at project, user, and system levels; environment variables (e.g., GEMINI_API_KEY); and command-line flags. The configuration subsystem resolves precedence across these sources and provides typed configuration objects consumed by Core, the Tool Registry, the Policy Engine, and the CLI layer.

Evidence: Understand shows a high fan-in on configuration modules, with import edges from client.ts, policy-engine.ts, tool-registry.ts, and multiple CLI modules. The breadth of these dependencies makes it a top-level architectural component that A1 should have identified. Its omission from A1 was a result of the documentation-based analysis not surfacing this cross-cutting concern.

Divergence 3: Session and Memory Persistence

A1 Conceptual View: A1 briefly mentioned that Gemini CLI "maintains persistent memory of projects" but did not represent session management or memory persistence as a distinct component or dependency in the architecture diagram.

Concrete Architecture: The codebase contains dedicated subsystems for session persistence: ChatRecordingService records conversation turns and tool results, ChatCompressionService handles token limit management by summarizing history when approaching context window limits, and a memory system (src/core/memory/) manages persistent project-level memories stored as files. These subsystems interact with both Core (for compression decisions during the agent loop) and the Host OS (for file I/O), creating dependency edges not present in A1's diagram.

Evidence: Git log shows the memory and compression modules have been present since early versions and are actively maintained. Understanding reveals that ChatCompressionService is invoked by GeminiClient when token counts approach model limits—a runtime dependency that shapes control flow in long conversations but was invisible in A1's conceptual model.

Divergence 4: CLI Frontend Dependencies Broader Than Expected

A1 Conceptual View: A1 showed the CLI Frontend with a single outgoing dependency to Core Runtime via the Core API/events interface. No other dependencies were indicated.

Concrete Architecture: Understand reveals that packages/cli has direct dependencies beyond Core. The CLI reads configuration files directly (not solely through Core), interacts with the Host OS for terminal capabilities and file system access (e.g., loading themes, reading .gemini/ directories), and subscribes to the MessageBus for tool confirmation UI. While A1 correctly identified the primary CLI → Core dependency, it underrepresented the CLI's involvement in the confirmation workflow and its direct access to configuration and filesystem resources.

Evidence: The Understand dependency graph for packages/cli shows import edges to configuration modules and to message-bus.ts in Core's confirmation-bus directory. The CLI subscribes to MessageBus events to render confirmation dialogs, making it an active participant in the safety workflow rather than a passive display layer.

Divergence 5: Sandbox Is an Execution Mode, Not an Architectural Boundary

A1 Conceptual View: A1's Figure 1 depicted sandboxing as a dashed isolation boundary encapsulating Built-in Tools, implying a distinct container or subsystem that wraps tool execution.

Concrete Architecture: Sandboxing in the concrete architecture is implemented as an execution mode within the tool execution pipeline rather than a separate architectural component. The ToolExecutor checks a sandbox flag and, when enabled, configures the subprocess to run within OS-level isolation (e.g., macOS Seatbelt sandbox profiles, Linux namespaces). There is no separate "Sandbox" module or subsystem with its own API surface; it is a configuration parameter applied at the point of process execution.

Evidence: Understand does not show a standalone sandbox subsystem. Instead, sandbox-related logic appears as conditional branches within tool-executor.ts and associated helper functions. This represents a minor divergence: A1's visual representation suggested more architectural separation than actually exists.

Architecture derivation process explained

We derived the concrete architecture by starting from our A1 conceptual split between the terminal-facing CLI and the backend core, then using SciTools Understand to recover the as-built structure from the extracted dependency file and source tree. We first grouped top-level files and folders into candidate subsystems based on package boundaries and dependency patterns, then iteratively reassigned files until the groups stabilized into the final concrete subsystems: CLI Frontend, Core Runtime, Tool Orchestration, Built-in File Tools, Memory, Policy & Confirmation, and MCP Support. This follows the A2 expectation that the report explain how the group moved from the A1 architecture to iteratively assigning code files and folders to those components, using Understand-based diagrams as the basis for the final redrawn architecture.

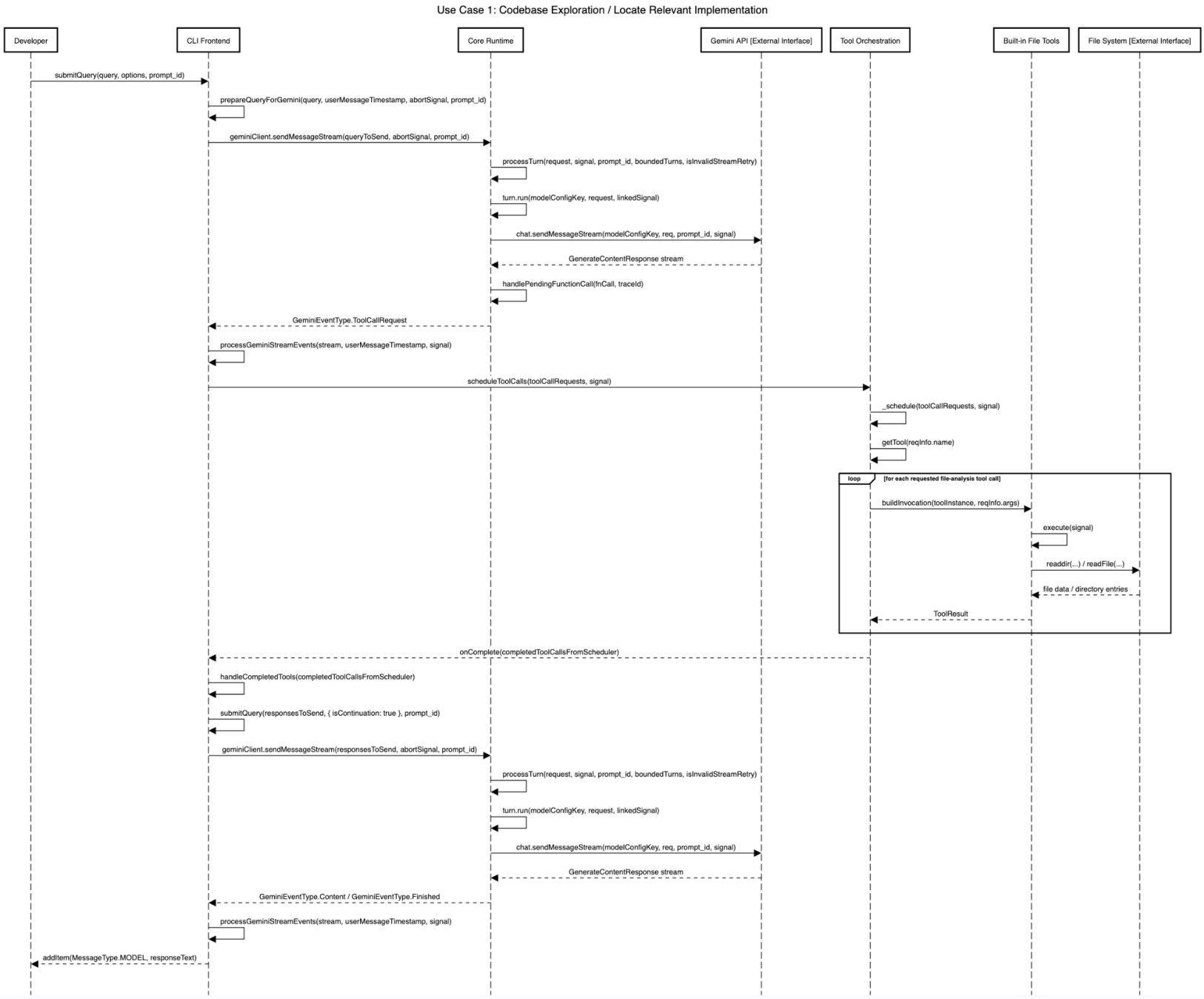
Understand was then used to validate the grouping by checking subsystem dependencies and tracing representative runtime paths rather than documenting low-level classes or procedures. In particular, we followed the main control flow for codebase exploration and documentation generation to confirm that requests move from the CLI Frontend into the Core Runtime, then into Tool Orchestration, Built-in File Tools, and Memory where applicable, with Gemini API and the File System treated as external interfaces. This kept the concrete architecture tightly linked to the A1 model while refining it into subsystem-level boxes and dependencies that match the recovered codebase and the two sequence diagrams.

Use Case Descriptions:

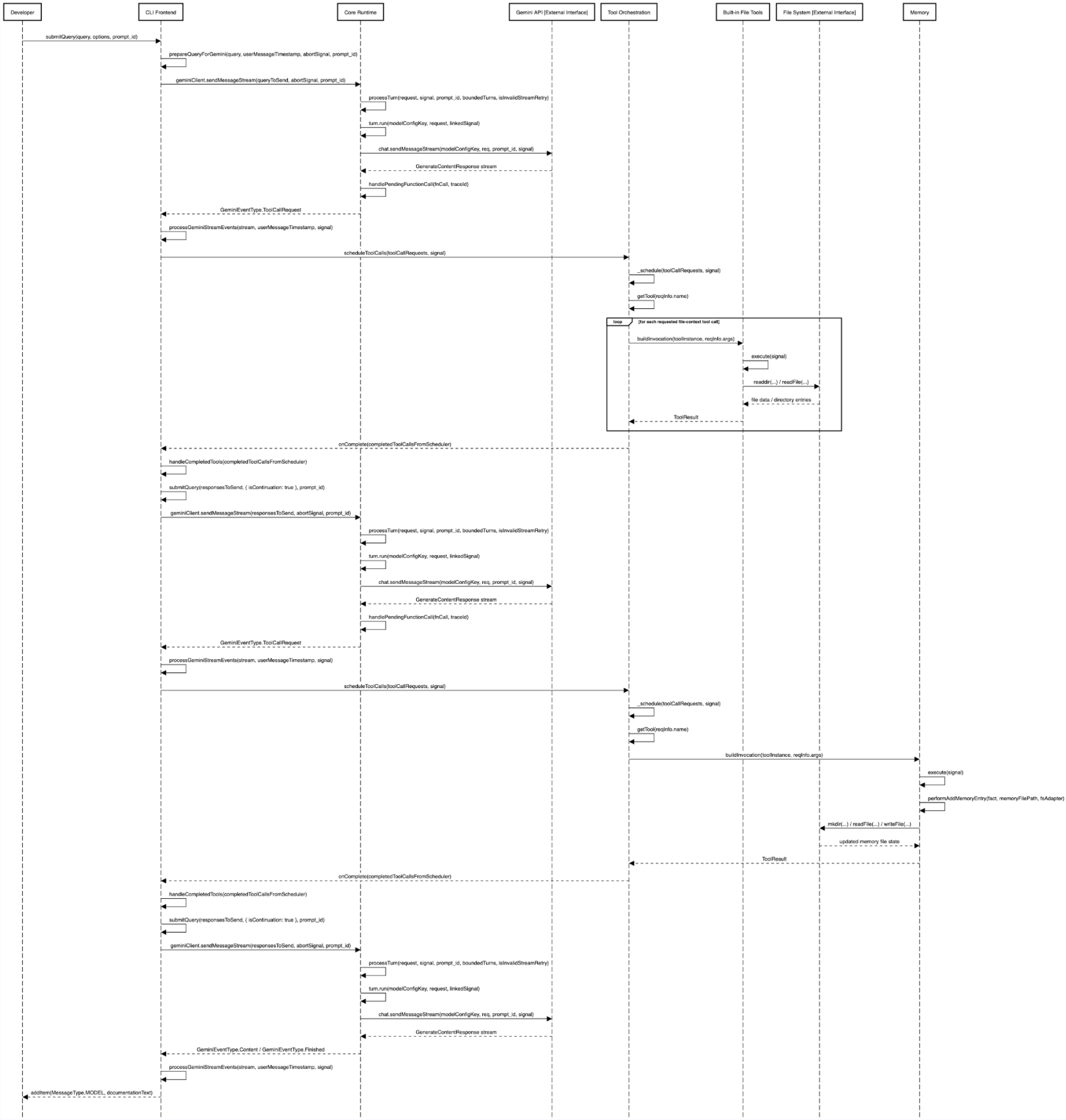
Use Case 1: This sequence diagram matches the concrete architecture because it follows the same dependency path shown in the high-level subsystem diagram: the request enters through the CLI Frontend, is passed to the Core Runtime, reaches the Gemini API for model reasoning, and then flows through Tool Orchestration to the Built-in File Tools, which interact with the external File System before control returns to the runtime and frontend. The diagram therefore reflects both the subsystem boundaries and the major dependency directions of the recovered concrete architecture, while grounding the interaction in concrete method calls rather than only abstract labels.

Use Case 2: This sequence diagram also matches the concrete architecture because it preserves the same primary control path from CLI Frontend to Core Runtime to Gemini API and Tool Orchestration, but extends the flow to the Memory subsystem before the final response is rendered. This makes it a useful complementary use case because it highlights an additional subsystem dependency not emphasized in Use Case 1, showing that the recovered architecture supports not only tool execution over the local file system but also persistence-oriented behavior within the same centralized orchestration model.

Sequence Diagrams for Use Cases



Use Case 2: Documentation Generation with Memory Persistence



External Interfaces:
At the high level, Gemini CLI interacts with three principal external interfaces: the Gemini API, the File System, and external MCP Servers. This matches the Level-1 concrete architecture, where the Core Runtime serves as the main coordinator for outbound communication and delegates specialized work to Tool Orchestration, Built-in File Tools, and related support subsystems. Framing the interfaces this way keeps the analysis aligned with the subsystem-level architecture rather than expanding the diagram with lower-level technical details.

The Gemini API is the primary external service used for model interaction. The Core Runtime sends prompts, conversation context, and tool declarations to the API and receives streamed model responses that may contain generated text or tool-call requests. This makes the Gemini API the main reasoning boundary of the system and confirms that model communication is centralized in the runtime rather than dispersed across the codebase.

The File System is reached primarily through the Built-in File Tools subsystem. These tools perform local codebase exploration and modification by reading files, searching content, and writing updates when appropriate. In architectural terms, this interface is important because it connects Gemini CLI's internal orchestration logic to the host development environment, enabling the system to act on local project artifacts rather than only produce text responses.

The MCP Server interface supports extensibility beyond the built-in toolset. Through MCP Support and Tool Orchestration, Gemini CLI can discover and invoke externally provided tools over supported transports. This confirms that external tool integration is not handled directly by the frontend or by a generic registry alone, but instead passes through the same controlled orchestration path used elsewhere in the concrete architecture.

Overall, these interfaces reinforce the central architectural finding of the report: external communication is funneled through a small number of controlled subsystem boundaries, preserving a predictable control flow while still allowing local execution and extensibility.

Lessons Learnt:
Analyzing Gemini CLI's architecture taught us to start with package structure before examining component details, as attempting to understand interactions without first mapping the monorepo boundaries caused initial confusion. We learned that production AI systems are fundamentally different from traditional applications: Gemini CLI uses 15+ specialized model configurations orchestrated together rather than being a simple API wrapper, a critical pattern we initially overlooked. Primary sources like official documentation and source code consistently proved more reliable than community blog posts, though secondary sources helped validate our interpretations. Maintaining the distinction between conceptual and concrete architecture required conscious effort, as we repeatedly caught ourselves documenting implementation specifics when the assignment required

subsystem level descriptions. Finally, studying the evolution strategy proved as valuable as understanding current design: the public roadmap's explicit priorities explained why the architecture emphasizes frontend backend separation and extensible tool systems. For future analyses, we will establish boundaries first, recognize domain specific patterns, prioritize authoritative sources, maintain proper abstraction levels, and always examine how systems are designed to evolve.

Conclusions:

This report analyzed the concrete architecture of Gemini CLI through subsystem recovery, dependency-based grouping, detailed examination of packages/core, and comparison against the Assignment 1 conceptual architecture. The analysis shows that Gemini CLI is organized around a centralized runtime that coordinates the frontend, model interaction, tool execution, memory, safety controls, and external integrations through a controlled orchestration flow. This architecture promotes maintainability and long-term evolution by combining centralized control with modular subsystem boundaries.

The recovered architecture confirms that Gemini CLI operates as a predictable, turn-based agent system in which user input is handled through the CLI Frontend, coordinated by the Core Runtime, and then, where necessary, extended through Tool Orchestration, Policy and Confirmation, Built-in File Tools, Memory, and MCP Support. Although the system supports asynchronous I/O, external processes, and extensible integrations, the overall control path remains deliberately structured in order to preserve auditability, safety, and user oversight. Approval gates and execution boundaries remain central to the architecture, especially for operations that can affect the local environment.

At the same time, the concrete architecture reveals a greater degree of internal specialization than was visible in the conceptual model from Assignment 1. In particular, the detailed analysis of packages/core shows that the runtime is not implemented as a monolithic backend, but as a collection of specialized internal elements for conversation management, tool scheduling, policy enforcement, and external coordination. The reflection analysis therefore shows that the conceptual architecture correctly captured the system’s major roles and control structure, while the concrete architecture exposes the finer-grained mechanisms required to support extensibility, safety, and maintainability in practice.

Overall, Gemini CLI’s concrete architecture demonstrates how a platform-oriented command-line assistant can remain both extensible and tightly controlled. Its subsystem structure allows the system to grow through additional tools, policies, memory mechanisms, and external integrations without losing the centralized orchestration model that gives its behavior predictability and coherence. This makes Gemini CLI a strong example of an evolving AI-assisted software platform whose concrete implementation remains aligned with its original high-level architectural intent.

AI Collaboration Report:

Model Selection - GPT-5.4, March 2026:

For this deliverable, our primary need was concrete architecture recovery and technical writing rather than large-scale code generation. Compared to Assignment 1, the emphasis shifted from conceptual architectural description to recovering the as-built structure of the system and comparing it against the earlier conceptual model. We selected GPT-5.4 because it provided a strong balance of codebase exploration, architectural reasoning, technical writing quality, and cost/latency appropriate for an analysis-heavy workflow.

Tasks Assigned to AI Teammate:

The prompts used in this assignment can be divided into the following three categories :
Codebase discovery: “Find X in the repo and show how it fits into the recovered subsystem structure”. The task was to locate where major architectural responsibilities live in the codebase, help group concrete files/modules into subsystems, and identify how those subsystems interact. This is high volume search work where AI speeds up locating relevant files and suggesting architectural groupings.

Draft verification against the repository and recovered architecture: “Review this section and tell us whether it matches the recovered concrete architecture.” The task was to review our draft text and either rewrite it for accuracy or flag claims that were too conceptual, too implementation specific, or insufficiently supported by the recovered subsystem structure. In both cases, the model was asked to point to relevant parts of the repository that confirmed or contradicted the wording. This helps identify mismatches between our written architecture description and what the codebase and subsystem structure actually support.

Academic writing support and reflection framing: “Rewrite this paragraph in a consistent report style and clarify how A2 differs from A1.” The task was to rephrase paragraphs into a consistent academic/report style, tighten language, standardize terminology, and help express the comparison between the Assignment 1 conceptual architecture and the recovered Assignment 2 concrete architecture. This improves clarity and cohesion efficiently while humans retain responsibility for technical correctness and final conclusions.

Interaction Protocol and Prompting Strategy:

Interaction with the AI was coordinated through a single designated prompt engineer (Mack Rabeau). Mack handled most prompts directly, then shared the AI’s outputs with the group for discussion. Other members contributed by proposing questions to investigate, reviewing whether the AI’s explanations matched our understanding of the architecture, and reviewing the AI’s academic writing for clarity, tone, and consistency with the report.

Our workflow was iterative and evidence driven. We typically started with a broad prompt to identify relevant subsystems, then followed up with narrower prompts that required traceability (specific files/modules and what role they played). Once we had the relevant sources, we drafted our own initial text based on the findings we had reviewed. We then used the model primarily as an accuracy check and revision aid: we asked it to critique our draft, flag any mismatches with the repository, and point to the specific areas in the docs/code that supported or contradicted our claims. This let us iteratively refine sections while keeping validation tied to primary sources.

Validation and Quality Control Procedures:

Any AI-assisted claim had to be checked against primary sources, the repository, and the recovered subsystem structure before it remained in the report. Our quality-control workflow consisted of source anchoring, draft verification against the codebase, and iterative revision. In practice, we used the AI to critique draft sections, then cross-checked the relevant code or documentation ourselves, and only kept AI-assisted text when the underlying architectural claim could be verified.

Quantitative Contribution to Final Deliverable

We estimate the AI contributed ~40% of the final submitted deliverable. This contribution was mainly in speeding up research and improving clarity, while humans remained responsible for correctness and final decisions (codebase/documentation discovery ~25%, draft critique for accuracy: ~10%, academic writing support: ~5%)

Reflection on Human–AI Team Dynamics

Using an AI teammate saved time during early exploration and drafting, but also created additional work in prompt iteration and validation. It improved brainstorming by quickly surfacing plausible architectural "places to look", but we treated these as leads and made final decisions as a group based on what we could verify in the docs and code. The main challenge was avoiding unsupported statements. We handled this by requiring file/document pointers and iterating until sections were correct. The biggest lesson we will apply in future projects is to use AI for search, synthesis and writing support, while enforcing a strict human verification gate (primary sources + team review) before anything enters the final report.

References:

Official Gemini CLI Architecture Documentation https://geminicli.com/docs/architecture/ Official architectural overview describing system design, component structure, and design patterns

Gemini CLI Architecture (GitHub) https://github.com/google-gemini/gemini-cli/blob/main/docs/architecture.md Detailed architecture documentation in the source repository explaining package structure and core components

Gemini CLI Public Roadmap https://google-gemini.github.io/gemini-cli/ROADMAP.html Development priorities, feature areas, and evolution strategy with explicit prioritization principles

MCP Server Integration Documentation https://google-gemini.github.io/gemini-cli/docs/tools/mcp-server.html Official documentation on integrating Model Context Protocol servers for external tool discovery

Gemini CLI npm Package https://www.npmjs.com/package/@google/gemini-cli Official npm package page with installation instructions, version history, and basic usage

"Gemini CLI: Agentic Software Engineering is Here" by C. Barkin Ozer https://cbarkinozer.medium.com/gemini-cli-agentic-software-engineering-is-here-f280c30e862f Community analysis discussing agentic capabilities and real-world software engineering applications

"Mastering the Gemini CLI" by Creative AI Ninja https://medium.com/@creativeaininja/mastering-the-gemini-cli-cb6f1cb7d6eb Tutorial and guide covering advanced usage patterns and workflow optimization

Google Gemini CLI Cheatsheet by Philipp Schmid https://www.philschmid.de/gemini-cli-cheatsheet Quick reference guide with common commands, configuration options, and usage examples

DeepWiki: Gemini CLI Documentation https://deepwiki.com/waywardgeek/gemini-cli Community-maintained documentation with additional examples and architectural insights

Gemini CLI Technical Comparison Discussion https://gist.github.com/steipete/670b2522b476f6a850e07b3535ebb2b7 Technical discussion comparing Gemini CLI with other AI coding assistants and architectural approaches

"An Introduction to Software Architecture" by Garlan & Shaw (1994) https://www.cs.cmu.edu/afs/cs/project/able/ftp/intro_softarch/intro_softarch.pdf Foundational paper defining architectural styles (layered, pipe-filter, event-driven, etc.) referenced in this analysis

"SAAM: A Method for Analyzing the Properties of Software Architectures" by Kazman et al. (1994) https://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.65.8786&rep=rep1&type=pdf SEI architectural analysis method required for Assignment 3

Model Context Protocol (MCP) Specification https://modelcontextprotocol.io/ Standard protocol for AI-tool integration that Gemini CLI implements for extensibility

Ink - React for Terminal Interfaces https://github.com/vadimdemedes/ink React renderer for terminals used by Gemini CLI's frontend for component-based UI

Google Gemini API Documentation https://ai.google.dev/docs Official documentation for the Gemini API that Gemini CLI wraps and orchestrates

Node.js Monorepos with npm Workspaces https://docs.npmjs.com/cli/v7/using-npm/workspaces Documentation on npm workspaces pattern used for Gemini CLI's monorepo structure